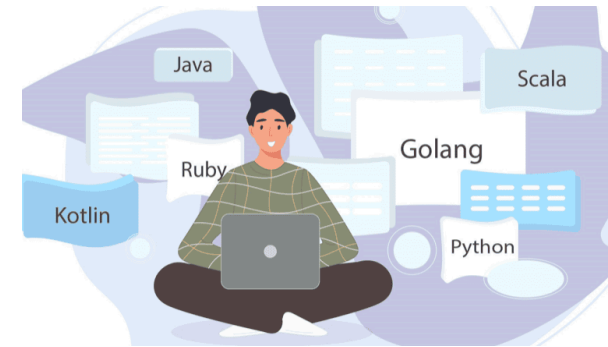
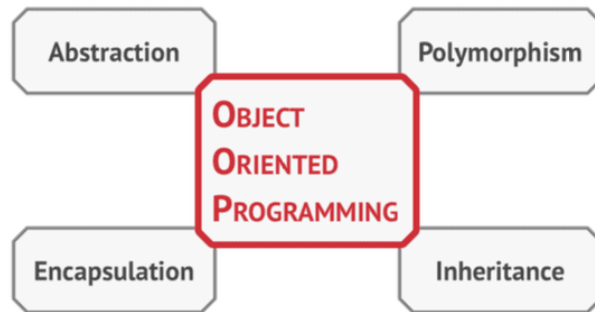


UML Diagrams

Structural Diagrams



- APIE forms the foundation of the OOP paradigm.
- Any OOP language can implement the four core OOP concepts.

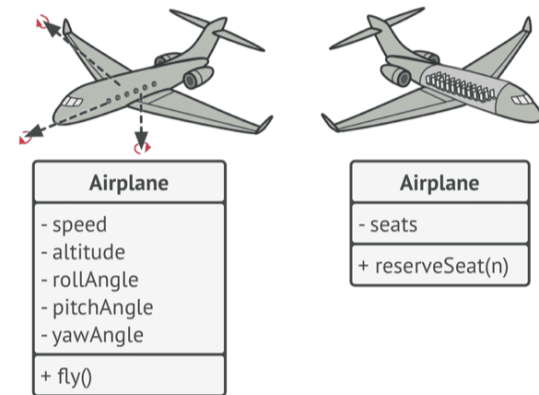
UML Diagrams



- UML (Unified Modeling Language) is the industry-standard visual language for OOP analysis and design.

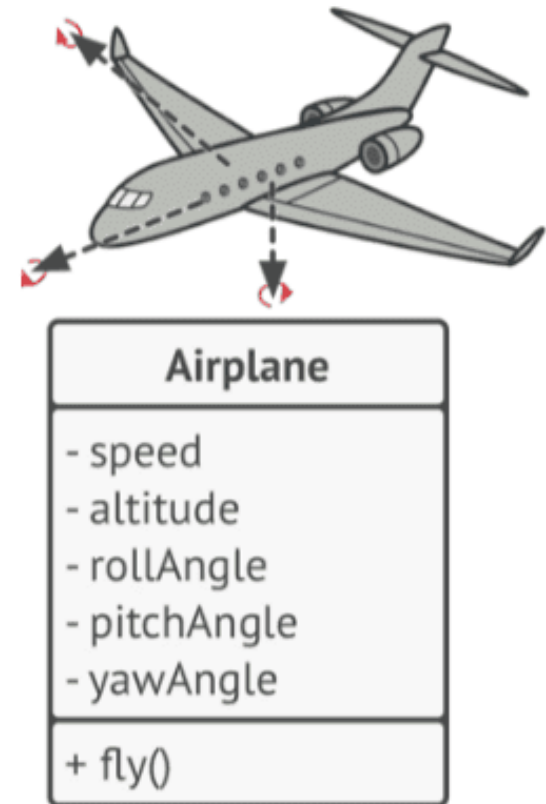
Class Diagram

- An airplane can be modeled as an object for flight simulation or reservations.

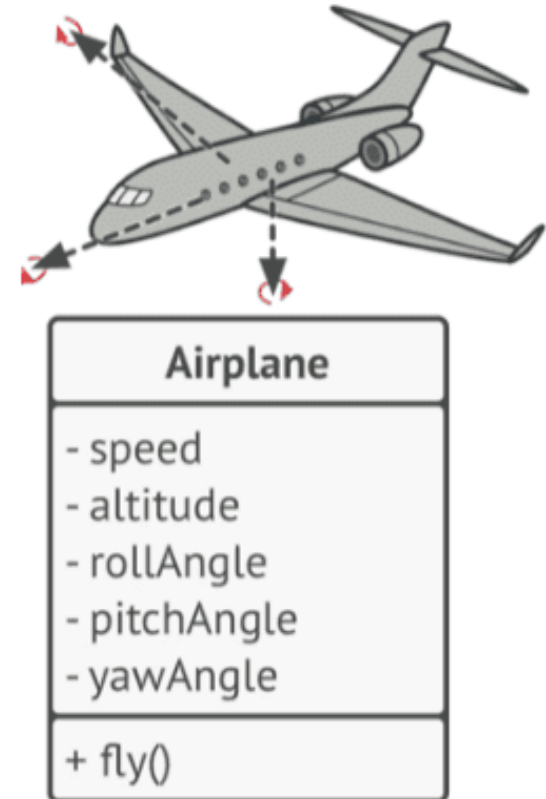


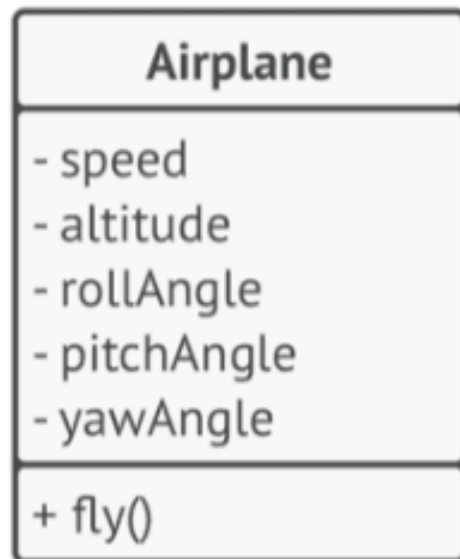
- A UML class diagram has three compartments:

- Class Name**
- Attributes** (state / fields)
- Operations** (behaviors / methods)



- Visibility notation:
 - `-` : Private
 - `+` : Public
 - `#` : Protected
- In typed designs, specify types (e.g., `- speed: int`, `+fly(): void`).



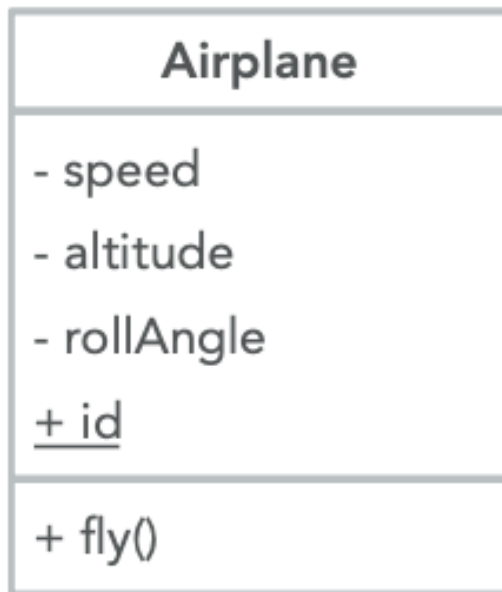


```
1 class Airplane(object):  
2     def __init__(self):  
3         self.speed = 0  
4         self.altitude = 0  
5         self.rollAngle = 0  
6         self.pitchAngle = 0  
7         self.yawAngle = 0  
8     def fly(self):  
9         print("I'm flying")
```

Class (Static) Attributes

- Attributes defined at the class level (outside methods) are class attributes.
- They are shared across all instances (note: `instance.id = ...` creates a shadowing instance attribute instead).

- In UML, static attributes and operations are indicated with an underline.



```
1 class Airplane(object):
2     id = 0 # class variable
3     def __init__(self):
4         self.speed = 0
5         ...
6
7 a = Airplane()
8 b = Airplane()
9 b.id = 10
10 print(a.id) # 10
```

Object Diagram

- A class is a blueprint; instantiating it creates objects at runtime.

```
class Airplane:  
    def __init__(self, speed=0):  
        self.speed = speed  
  
a = Airplane(speed=100) # object instantiation
```

UML object diagrams capture a runtime snapshot:

1. **instanceName : ClassName** (underlined header)
2. **Attributes** with their current values assigned

Python code to Object Diagram

a: Airplane

speed = 10
altitude = 20
rollAngle = 30

B: Airplane

speed = 100
altitude = 200
rollAngle = 300

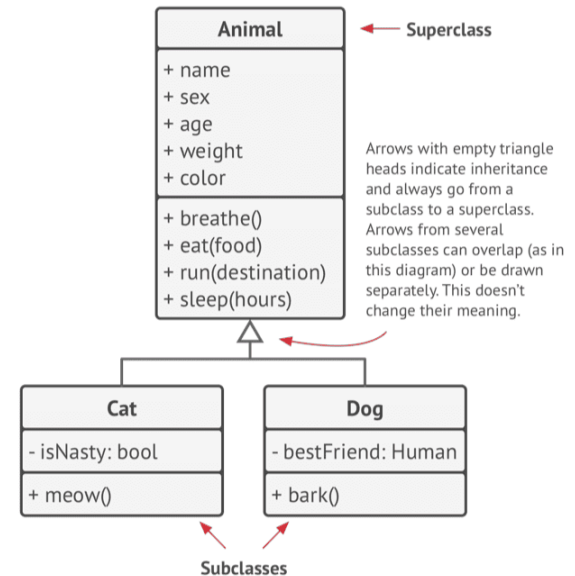
```
1 a = Airplane(10, 20, 30)
2 b = Airplane(100, 200, 300)
```

Relationship among classes

- UML describes a single class or object using class and object diagrams.
- UML can describe the relationship among classes.

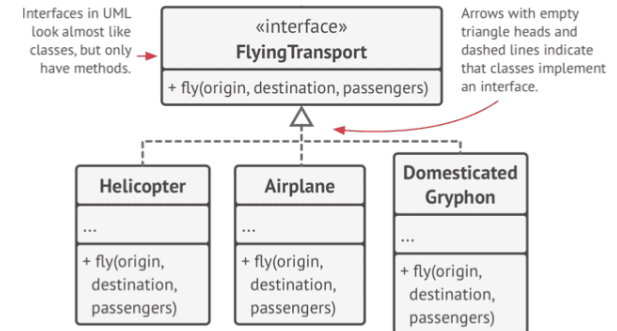
Inheritance

- In Python, extend a class by listing the superclass in parentheses.
- In UML, show it with a solid line and a hollow arrowhead pointing from subclass to superclass.



Interface

- In UML, interface implementation (realization) uses the same hollow arrowhead as inheritance, but with a dashed line pointing from the implementing class to the interface.
- The dashed line is what distinguishes



Composition (Strong Whole-Part)

- **Strong ownership & coincident lifecycle:** The whole creates and owns the parts.
- When the whole is destroyed, its parts are disposed of too (if no external references exist).
- UML symbol: **Solid (filled) diamond** on the whole side.

```
class Department:
    def __init__(self, name):
        self.name = name

class University:
    def __init__(self, name):
        self.name = name
        # Departments are instantiated and owned internally
        self.departments = [
            Department("Computer Science"),
            Department("Software Engineering")
        ]

u = University("NKU")
```



- Structurally, a University owns its Departments; they are not meant to exist independently.

Aggregation (Weak Whole-Part)

- **Shared ownership & independent lifecycle:**
The part can exist independently of the whole.
- A Professor can exist before joining a University and stays valid after leaving.
- UML symbol: **Empty (hollow) diamond** on the whole side.

```
class Professor:
    def __init__(self, name):
        self.name = name

class University:
    def __init__(self, name):
        self.name = name
        self.professors = []

    def hire_professor(self, prof):
        self.professors.append(prof)

# Professor exists independently
p = Professor("Dr. Cho")
u = University("NKU")
u.hire_professor(p)
```



- In Python syntax, both use references (e.g. lists).
- The key difference is **architectural**: ownership and lifecycle management.

| Dependency ("Uses-A")

- **Temporary usage relationship:** One class briefly uses another (e.g., as a parameter, return value, or local variable).
- Changes to the supplier class may affect the client class.
- UML symbol: **Dashed line with an open arrowhead** pointing to the used class.

```
class Salary:
    def __init__(self, amount: float):
        self.amount = amount

class Professor:
    def __init__(self, name: str):
        self.name = name

    def receive_salary(self, salary: Salary):
        # Uses Salary as a parameter without storing it
        print(f"{self.name} received ${salary.amount}")

s = Salary(5000)
p = Professor("Dr. Cho")
p.receive_salary(s)
```



- **Professor** depends on **Salary** as a parameter to perform an operation.
- This is a weaker, non-structural coupling compared to Association.